CSSE 230 – Data Structures and Algorithms

Exam 3 Winter 2022-23 Name: _____SOLUTION_____ Section: 01 02 03 04

You may use a calculator, a writing implement, and a 1-sided 8.5" x 11" note page. You must turn in this written part before you use your computer, textbook, or any other resources.

You may not communicate with any person other than your instructor about this exam until allowed by your instructor. You may not use electronic devices.

Part 1 scores (for instructor use):

Problem	Possible	Score
1	10	
2	9	
3	14	
4	14	
5	18	
Total	65	

Computer part = 55 points

Total	**120**	

Part 2 (computer) rules

You may use any printed or written materials that you brought with you, code that you wrote yourself or with a partner before the exam, anything on the CSSE230 Moodle and web sites, and the Java API documentation (on your laptop or at oracle.com).

You may not otherwise search the internet or communicate with any person other than your instructor or a student assistant. You may not communicate with anyone else about this exam until allowed by your instructor. You may not use other electronic devices.

You will sign a statement to certify that you neither received help from others nor gave it on either part of the exam.

You must get these problems working on your computer. All or most of the credit for these problems will be for code that actually works.

1. (10 points) Below you will find several statements. A statement is true (T) if it is always true. It is false (F) if there is at least one counterexample (sometimes false). **Circle at most one answer for each part.**

   a. T (F) In order to sort an array of integers into ascending order, we must use a min heap when using the Heapsort algorithm

   b. T (F) Consider an in-order traversal of a BST, this algorithm has O(N) performance on an *unbalanced* BST, and it has O(log(N)) performance on a *balanced* BST

   c. (T) F If a hash table is implemented using the separate chaining technique, every entry of the hash table's array stores either a reference to a node, or it stores null

   d. T (F) In Figure 1, root's balance code will either be \ or / after a single insertion into either of root's subtrees

   e. T (F) Inserting a node in a subtree sometimes requires calling a size() method to recalculate the size.

Figure 1

2. (9 points) You are given a Node class (as shown below) for a height-balanced BinaryTree T that holds character data:

```
class Node {
    char data;
    Node left;
    Node right;
    int rank;
}
```

To Do:

Implement the following method whose job is to insert the new character at the front of the EditorTree's edit string, i.e., this is the same as doing: `add(item, 0);` Assume that this method is written in the Node class. It will be called on the root, like: `root = root.cons(item);` The method should handle updating the ranks due to the insertion, but you don't need to worry about updating balance codes or rank changes that result from rotations (the call to `balanceAfterInsert`, handles those changes).

Note: Your implementation for *cons* is not allowed to call the *add* operation, it is required to directly manipulate the tree's nodes and the fields of those nodes. You may assume NULL_NODE exists.

```
class Node {

   …
   Node cons(char item) {


   if (this == NULL_NODE) {
         return new Node(item);
   } else {
         // navigate to left most NN
         this.left = this.left.cons(item);
         this.rank++;
         return this;
   }


   return balanceAfterInsert(); //rebalance now that item has been inserted
} // end cons
```

3. (14 points) EditorTree balancing. Consider the following diagram in Figure 2.

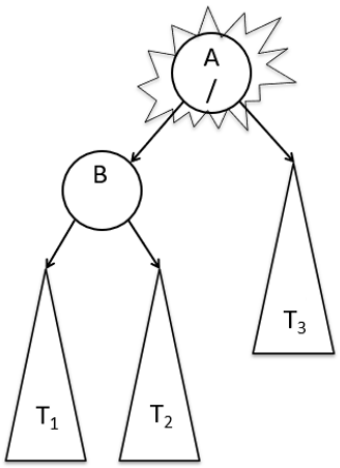


Figure 2

Explanation:

- [Before deletion] The tree was height-balanced and A's balance code was LEFT (/)
- One node was deleted.
- Balance codes were updated, starting at the point where a node was deleted, and moving up the tree.
- [Imbalance detected] The lowest imbalance is found at A, as pictured.
- Heights of T_1, T_2, and T_3 are not necessarily drawn accurately.

Fill in the blanks below.

a. (2 points) In Figure 2 (above), in which tree did the deletion occur? Circle one: T_1 T_2 (circled) T_3

b. (2 points) In Figure 2 (above), at the time the imbalance is detected, B's balance code showed that a single-right rotation is the only possible rotation needed at A; B's balance code must have been ___/ ____

c. (4 points) We then perform a rotation, leading to the diagram in Figure 3 (to the right)

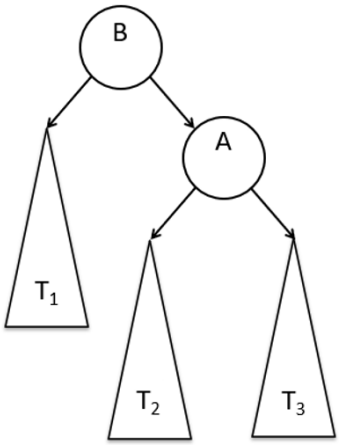


Figure 3

i. What is the new balance code of A? __=__

ii. What is the new balance code of B? __=__

d. (4 points) Suppose R_A, R_B are the "old" ranks of A and B before starting deletion. Using these, give formulas for the "new" ranks of A & B (in Figure 3) after the deletion and rotation.

i. What is the new rank of A? ___ $R_A - (R_B + 1)$ _____

ii. What is the new rank of B ? ____ R_B _____

e. (2 points) Suppose A was originally the root of a subtree—it had a parent, say P. Does P need to change its balance code after this rotation? Circle one:

(Yes) No

4. (14 points) For this problem, consider a hash set where finding the initial probe index into the table the hash code is taken modulo the array capacity M.

(a) (2 points) Suppose we are using **quadratic probing** on an array where M = 100. If we are inserting a new element that hashed to int value 932498, what are the first four indices where we attempt to insert?
(Your answers should be integers between 0 and 99)

___98____ ___99____ ___2____ ___7____

(b) (2 points) Suppose we are using **linear probing** on an array where M = 100. If a new element is hashed to int value 856235, what are the first four indices where we attempt to insert?

____35____ ____36____ ____37____ ____38____

(c) (10 points) Suppose that we are using separate chaining on a hash table whose array's M = 10, and we want to maintain a load factor of $\lambda \leq .60$ by doubling capacity when needed.

i. (3 points) What is the big-θ runtime of adding 1 element, assuming a good hash code function?
_____**Amortized $\theta(1)$ and worst-case $\theta(N)$**_____

ii. (3 points) What is the minimum number of items required to add that would cause the array to grow in capacity the first time? ___7____

iii. (4 points) Suppose we need to double the capacity of the hash table shown in Fig. 4 to the hash table shown in Fig. 5, where each item has the following hash codes:
("hello": 40), ("ciao": 58), ("hola": 60), ("salut": 43), ("bonjour": 71)

To Do: correctly insert (by diagramming) the nodes shown in Fig. 4 into the hash table in Fig. 5

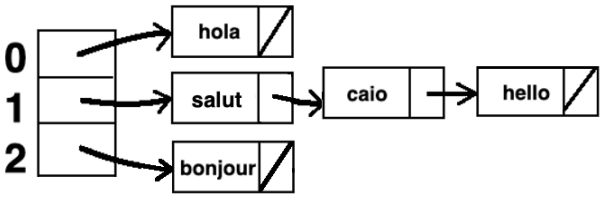


Figure 4

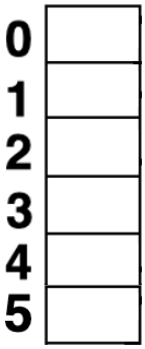
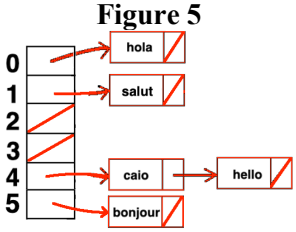
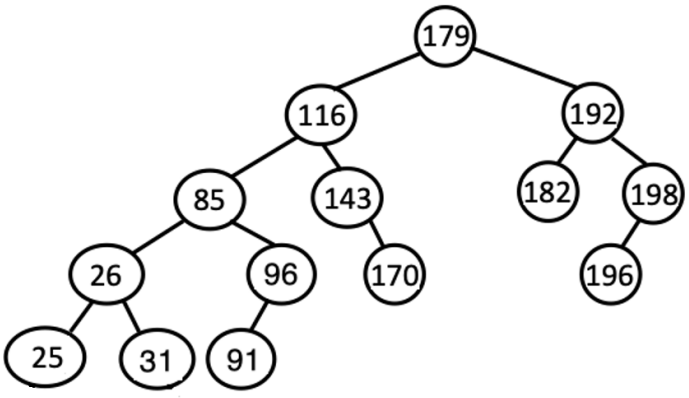


Figure 5



5. (18 points) Consider the AVL tree below.
For parts (a) – (c) fill in the table below. For each part, **begin with the original tree**.



			single, double, or none?	left or right?	root of rotated subtree before rotation (i.e., "imbalanced node")	root of rotated subtree after rotation
Example	insert 30	1st rotation	single	right	116	85
		2nd rotation	none			
part (a)	insert 175	1st rotation	**single**	**left**	**143**	**170**
		2nd rotation	**none**			
part (b)	delete 170	1st rotation	**single**	**right**	**116**	**85**
		2nd rotation	**none**			
part (c)	delete 182	1st rotation	**double**	**left**	**192**	**196**
		2nd rotation	**single**	**right**	**179**	**116**

Reminder: did you start with the original tree for each part?

Work area:

Programming Solution

BST Class

```
    /**
     * See the written document for more details
     */
    public int numHeightAndDepthBothOddOrEven() {
        // TODO Write this!
        return this.root.numHAndDOddOrEven(0).count;
    }

    //Container Class
    private class HeightAndCountCont {
        int height = -1;
        int count = 0;

        HeightAndCountCont(int h, int s, int c) {
            this.height = h;
            this.count = c;
        }
    }
```

```
    public BinarySearchTree(List<Integer> nodeData) {
        // TODO: Implement this constructor.

        nodeData.sort(null);

        //No need to check if nodeData exists
        int midIndex = nodeData.size()/2;
        this.root = new Node(nodeData.get(midIndex)); // Start in the middle

        //We split this out into two recursive calls since they operate differently
        this.root.left = makeFlyingGeeseLeft(nodeData, midIndex - 1); //Start just to the left of middle
        this.root.right = makeFlyingGeeseRight(nodeData, midIndex + 1); //Start just to the right of middle

    }

    private Node makeFlyingGeeseLeft(List<Integer> nodeData, int index) {
        if (index < 0) { //Don't go less than 0
            return NULL_NODE;
        } else {
            Node toCreate = new Node(nodeData.get(index));
            toCreate.left = makeFlyingGeeseLeft(nodeData, index - 1);
            return toCreate;
        }
    }

    private Node makeFlyingGeeseRight(List<Integer> nodeData, int index) {
        if (nodeData.size() == index) { //Don't go past the end of the array
            return NULL_NODE;
        } else {
            Node toCreate = new Node(nodeData.get(index));
            toCreate.right = makeFlyingGeeseRight(nodeData,index + 1);
            return toCreate;
        }
    }
```

Node Class

```
private HeightAndCountCont numHAndDOddOrEven(int depth) {
    if (this == NULL_NODE) {
        return new HeightAndCountCont(-1, depth, 0);
    }
    HeightAndCountCont leftCont = left.numHAndDOddOrEven(depth+1);
    HeightAndCountCont rightCont = right.numHAndDOddOrEven(depth+1);
    int myHeight = Math.max(leftCont.height, rightCont.height) + 1;
    int myCount = leftCont.count + rightCont.count;
    myCount += (myHeight % 2 == depth % 2) ? 1 : 0;
    return new HeightAndCountCont(myHeight, depth, myCount);
}
```